

First Deliverable - A CSR SpMV parallelization study

Vittoria Zandonà

ID: 244816

GitHub Repository: <https://github.com/fumyskin/PARCO-Computing-2026-244816.git>

Abstract

This report is a case study of different parallelization techniques for the classical CSR SpMV algorithm. Based on an already existing and verified implementation, several approaches of experimental code with OpenMP pragma statements are going to be tested on different real world sparse matrices to evaluate information regarding speedup, cache efficiency, bandwidth and potential scalability. The simulations are going to be tested on different architectures on the HPC cluster, which is NUMA based. Furthermore, the final results will be compared to the performance of the state-of-the-art CSR Merge-based SpMV to understand even more deeply the limitations of our initial approach.

Keywords: CSR, OpenMP, Merge-based algorithm, Parallelization

1. Introduction

The remainder of this report is organized as follows: Section 2 introduces the CSR format and the CSR classical SpMV algorithm, Section 3 defines the main parallelization challenges that will need to be addressed in our implementation. Section 4 describes the experimental setup, Section 5 describes the methodology. Section 6 discusses the results obtained. Section 7 discusses the Merge-Based algorithm state-of-the-art. Section 8 offers some concluding remarks.

2. CSR format and CSR SpMV algorithm

The CSR format has become one of the standard algorithm for matrix storage formatting of sparse matrices [1]. This format stores the nonzero values of the rows of a given sparse matrix in consecutive memory locations, and it stores an index to the first stored element of each row. In the reference implementation we will follow [2] the CSR translates the COO format matrix by keeping the *values*[*nnz*], *col_ind*[*nnz*] and *row_ptr*[*n*] to store each nonzero matrix entry's coordinates. Specifically, the *row_ptr* array stores the index of each row in *values*, i.e., if *values*[*k*] stores matrix element $a_{i,j}$, then $\text{row_ptr}[i] \leq k < \text{row_ptr}[i + 1]$.

The *col_ind* array stores the column indices of the elements in the *values* array, that is, if *values*[*k*] stores matrix element $a_{i,j}$ then *col_ind*[*k*] = *j*. To begin our analysis, we define the pseudocode of the serial version of one of the classical versions of the SpMV algorithm. (See algorithm 1) [3] - [4].

3. Parallelization Challenges

Before describing the parallelized solutions it is important to define what are the critical points that need to be either minimized or solved totally in our final solutions to see concrete results. Our main concerns are about:

1. Scalability and potential bottlenecks and race conditions
2. Cache performance under parallelization (in particular, avoiding false sharing)

Algorithm 1: Sparse matrix-vector multiplication (CSR format)

```

for i ← 1 to n do
    sum ← 0;
    for k ← row_ptr[i] to row_ptr[i + 1] − 1 do
        | sum ← sum + val[k] · x[col[k]];
    end
    y[i] ← sum;
end

```

3. Load balance
4. Memory and computational bandwidth bounds

3.1 Race Conditions, Scalability and Bottlenecks

When parallelized with respect to rows, algorithm 1 in the figure does not suffer from race conditions, because each parallel iteration *i* writes to a single location *y*[*i*] which is not updated by any other iteration. For parallelizing such algorithm, we are concerned with how it scales with the number of processors. To measure the complexity of the code given we start by considering the following two parameters [3]:

1. The **work**, denoted T_i , the running time on a single processor
2. The **critical-path length** or **span**, denoted T_∞ , the running time on an infinite number of processors. We can think of it as the length of the longest sequence of operations that cannot be executed in parallel.

The **parallelism** of the algorithm is defined as T_i/T_∞ and corresponds to the maximum speedup on any number of processors [3]. The work of Algorithm 1, assuming that $nnz \geq n$, is $\Theta(nnz)$: the outer loop executes for *n* iterations and the inner loop executes all the nonzero values for each i_{th} row, thus for a total of *nnz* times.[3] The critical-path length depends on the maximum number *nr* of nonzeros in any row of the matrix

A, since that number determines the worst-case time of any iteration of the loop. The n iterations of the parallel loop in the outer loop contribute $\Theta(lgn)$ to the span. The final result for the parallelism is $\Theta(nnz/(nr + lgn))$. Nevertheless, we need to consider two corner cases: the first one is the case $nr = O(1)$, which is the case in which no row is long. Then parallelism is $\Theta(nnz/lgn)$ and the scalability is high. Conversely, if even a single row of the sparse matrix is dense, $nr = \Theta(n)$ and the parallelism becomes $nnz/n = \Theta(1)$ and that single row becomes the bottleneck: the work is equal to the span and there is no possibility of parallelism. Solutions to this problem have already been found: parallelizing the inner loop in the right way can reduce the sequential inner product cost from $\Theta(nr)$ to $\Theta(lgnr)$, which brings great asymptotic improvement [3]. However, such solutions are difficult to apply efficiently with standard OpenMP library tools. Even if we attempt to emulate the suggestions using a parallel reduction tree via standard loop constructs (eg, using `#pragma omp parallel for reduction(+:sum)`), the work would be the same, but the span does not become $\Theta(lgnr)$ in a practical execution environment. A standard OpenMP reduction partitions the iteration space among T threads. Each thread processes a contiguous block sequentially ($span \simeq nr/T$) then the reduction combines T partial sums ($span \simeq lgT$) so the total span would actually be $\Theta(nr/T + lgT)$. To get $\Theta(lgnr)$ span, we would need to have $T = \Theta(nr)$ threads, which is unfeasible. Furthermore, while for example OpenMP *tasks* could theoretically model the recursive divide-and-conquer tree, the runtime overhead of spawning thousands of tasks for a single row usually outweighs the arithmetic benefits for typical matrix sizes. Thus, OpenMP does not easily solve the dense-row bottleneck without incurring prohibitive synchronization costs.

Apart from bottleneck risks, as cited in Intel documentation (<https://tinyurl.com/59wdz6tx>) the main problems that must be taken into considerations when using OpenMP are **scheduling overhead** and **imbalance barriers**. In irregular workloads, static distribution often results in significant idle time at synchronization barriers, as threads must wait for the slowest thread to complete. While dynamic scheduling mitigates this by assigning work on-demand, it introduces runtime scheduling overhead, particularly when processing fine-grained chunks. Consequently, optimization requires navigating the trade-off between minimizing load imbalance and limiting scheduling costs, a balance we empirically explore in Section 4.

3.2 Cache and Memory Performance

The SpMV kernel suffers from irregular memory access patterns, specifically due to the indirect addressing of the vector x via $col[k]$. This unpredictability defeats hardware prefetching mechanisms, leading to frequent cache misses. On shared-memory architectures, this is compounded by **false sharing**, where threads concurrently modify data residing on the same cache line. This triggers cache coherency protocols that invalidate the line and force expensive re-loads, significantly degrading parallel efficiency. To mitigate this, we explore

both cache-oblivious and cache-aware alignment strategies [5]. Furthermore, SpMV exhibits inherently low arithmetic intensity (the ratio of FLOPs to data transfer)[6]. As a result, the kernel is **memory-bound**: performance is limited by memory bandwidth rather than computational throughput. Due to the low computation per loaded byte, the processor cannot effectively hide memory latency, causing stall cycles to accumulate during remote memory fetches.

3.3 Bandwidth and Arithmetic Intensity

To quantify the bottleneck, we analyze the kernel's arithmetic intensity (I) against the hardware's machine balance. The SpMV inner loop performs 2 FLOPs per nonzero. Data traffic includes fetching the matrix value (8 bytes), the column index (typically 4-8 bytes), and the vector element x (8 bytes). Neglecting the amortized cost of row pointers and output stores, this requires approximately 24 bytes per iteration.

$$I \approx \frac{2 \text{ FLOPs}}{24 \text{ Bytes}} \approx 0.083 \text{ FLOPs/Byte}$$

Using the system specifications ($BW_{mem} \approx 563.2 \text{ GB/s}$ and $BW_{math} \approx 7.06 \text{ TFLOPS}$), we calculate the machine balance point:

$$\frac{BW_{math}}{BW_{mem}} \approx \frac{7060 \text{ GFLOPS}}{563.2 \text{ GB/s}} \approx 12.5 \text{ FLOPs/Byte}$$

Since $I \ll 12.5$, the algorithm is effectively situated far to the left of the Roofline ridge point, proving it is heavily **memory-bound**. Consequently, scaling is limited not by compute availability, but by the saturation of memory controllers and cross-socket interconnects in the NUMA hierarchy.

3.4 Processor Affinity and Locality

Given the memory-bound nature of the kernel, optimization must prioritize minimizing data movement and maximizing locality. To achieve this, we employ **thread pinning** (processor affinity) to bind threads to specific cores. This strictly prevents the operating system from migrating threads across NUMA sockets, which would otherwise incur severe latency penalties due to remote memory accesses and cold cache effects. However, while pinning secures locality, it disables the OS's ability to dynamically rebalance threads, making the application solely responsible for handling **load imbalance** via efficient work distribution strategies. We will hence treat different cases of work balance OpenMP strategies to solve this problem.

4. Experimental setup

For the experimental part we run our simulation jobs on a short queue (short_cpuQ) of our available HPC cluster. The queue emulates an Intel Xeon Gold 6252N CPU, a NUMA x86_64 architecture with 96 CPU cores clocked at 2300 MHz. The compiler used is gcc version 9.1.0. The organization of the code found in the repo can be organized as follows:

- **experiment0.c** is the baseline CSR Spmv sequential code which implements.
- **experiment1.c** is the first parallelization attempt with pragma OMP. In here only the outer loop is parallelized with different scheduling techniques, namely: static, dynamic, guided, auto, runtime and for.
- **experiment2.c** attempts to use pragma construct to manually divide work between threads and attempts to reduce the problem of false sharing by introducing cache alignment for allocating matrices and vectors.
- **experiment3.c** attempts to write a work division solution using task pragma method.
- **experiment4.c** is a fused solution between exp1.c and exp2.c: it tries to minimize false sharing by using cache alignment while using the best performing scheduling solution that has been found in previous simulations. Furthermore, it attempts to assure thread pinning by using the `PROC_BINDING` construct when run from bash terminal.
- **experiment5.c** is an attempt to implement an alternative state-of-the-art CSR based algorithm (merge based algorithm)

5. Methodology

All the algorithmic approaches are going to be **fixed row-based partitioning**, ie, we are going to divide work by rows. All the results are going to be compared together on 11 different matrices taken from the *SuiteSparse Matrix Collection*, with different sparsity levels (the chart for matrices is found in the repo at `.../MATRICES/matrix_char.png`). The testbenches have been run in two ways:

- the first set of testbenches removes the overhead from cold start by running in the main of each executable 20 times.
- The second set of testbenches reproduces the same simulations but with the flags
`-O3 -fopenmp -ffp-contract=fast` to improve branch predictions.

The images of the testbenches for each matrix can be found in the repository.

6. Experimental Results and Discussion

6.1 Scalability and NUMA Topology

The experimental data reveals a consistent scalability pattern across all matrix types: performance improves linearly up to approximately 32 threads before degrading explicitly at 64 and 96 threads. This inflection point might correlate directly with the hardware topology: the test system comprises four sockets with 24 cores each. Spanning computation beyond a single socket (> 24 threads) forces memory traffic to cross inter-socket links. Since SpMV is memory-bound, the latency and bandwidth saturation of these links outweigh the benefits of additional compute cores, causing the observed performance collapse. Thread creation overhead was ruled out as a cause, as OpenMP threads are instantiated only once. Another contributing factor may be load imbalance and increased coordination overhead at high thread counts. As the number

of threads grows, the amount of work per thread becomes smaller, and the irregularity of the sparse matrix structure may cause some threads to finish earlier than others. The latter overhead is directly proportional to $O(T)$, where T is the number of threads. The resulting synchronization and scheduling overhead becomes proportionally more significant and can further degrade performance. This heavily depends on the type of matrix, so it may explain lack of scaling only in specific cases.

6.2 Scheduling Strategies

In `experiment1.c` we evaluate 3 main OpenMP scheduling:

- **Static Scheduling:** consistently outperformed other methods for regular matrices. By deterministically mapping contiguous row blocks to threads in round-robin fashion, it maximizes memory locality and eliminates runtime scheduling overhead.
- **Dynamic Scheduling:** assigns small chunks of work to threads as they become idle. Although this approach is beneficial when iteration costs vary widely, its runtime overhead is considerably higher, especially when the default chunk size is 1. More importantly, dynamic scheduling disrupts memory locality: threads frequently jump to distant memory regions, causing extra cache misses. In our tests, where most matrices are relatively regular, these disadvantages outweigh any potential load-balancing benefit, so dynamic scheduling rarely improves performance. It only proved beneficial when coupled with medium chunk sizes (100 – 1000) on highly irregular matrices.
- **Guided Scheduling:** Emerging as the optimal compromise for irregular datasets (e.g., *cond-mat-2005*, *TSOPF*), guided scheduling effectively balanced load distribution against overhead by tapering chunk sizes as execution progressed. In terms of performance, it's very similar to static scheduling. Our findings are coherent with the conclusions of others: [5].

Chunk size optimization followed a convex curve. Small chunks (< 10) caused excessive synchronization overhead behavior in the scheduler. Large chunks (> 10,000) effectively reverted the execution to a coarse-grained static decomposition, starving threads of work (load imbalance) as the number of chunks became insufficient for the available cores ($n_{chunks} < n_{threads}$). The optimal window was consistently found between 100 and 1000 iterations.

6.3 Impact of Optimizations: Alignment and Affinity

`experiment2.c` is an attempt to decrease false sharing by introducing the function `posix_mem_align` function. In this way, the dense vector and the resulting vector are allocated in memory at memory addresses which are multiple of 64 bytes, which is the standard size for a cache line. Furthermore, each thread gets a contiguous block of rows through the manual work division obtained by defining `start` and `end` variables, which is good for cache locality and avoiding false sharing in the output vector `p`. The manual implementation, with respect to static

scheduling, guarantees alignment with thread boundaries and avoids dependence on OpenMP implementation. Since it theoretically subdivides work as the static scheduling does, the trend for both manual and static are almost the same for every matrix, but manual proves to be consistently faster, even by little, with respect to the static performance (+2-5% higher than static performance in terms of speedup).

`experiment3.c` instead of statically assigning contiguous rows to threads, it creates one OpenMP task per row, allowing dynamic load balancing at the cost of higher overhead. This simulation consistently proves to be the most inefficient.

`experiment4.c` applies thread pinning via the `PROC_BIND` command [7]. The results reveal a distinct trade-off between peak performance and stability. In the optimal thread range (16-32), pinning yielded slightly lower speedups compared to the unpinned `static` or `manual` schedules. This suggests that strict core binding prevents the OS from mitigating minor load imbalances, forcing the entire team to wait for slower threads. However, regarding scalability at high thread counts, pinning proved superior. While the unpinned strategies suffered a drastic performance collapse after 64 threads (likely due to cross-socket thread migration and cache invalidation), the pinned simulation maintained stable execution times. Thus, while pinning sacrifices slight peak performance, it acts as a critical guardrail against NUMA-induced performance degradation. Figure 1 shows the plot for `cond-mat-2005.mtx` with all the execution times compared. ^a

7. Comparison with State-of-the-Art: Merge-Based SpMV

Standard CSR parallelization strategies that attempt to take advantage of rows independence to parallelize often suffer performance degradation on matrices with irregular row lengths or wide aspect ratios[8]. While `schedule(guided)` offers some mitigation, it does not strictly decouple the workload from the non-zero distribution[4]. To address this, we attempt to implement the heuristic of the Merge-Based SpMV algorithm, which is a modern alternative to the classical Spmv CSR algorithm [1]. This approach conceptualizes SpMV as a logical merge of two lists: the row pointers (horizontal moves) and non-zero indices (vertical moves). By using a "merge-path" search, the algorithm calculates starting points that ensure every thread processes an equal total number of coordinates (rows + non-zeros), guaranteeing perfect load balance regardless of matrix structure. On our standard test suite, the Merge-Based implementation yielded results worse to `schedule(guided)` and `schedule(static)`, primarily because the test matrices lacked sufficient sparsity variation to justify the algorithmic overhead. To stress-test the algorithm, we utilized the `termomech_dK`, `ASIC_320k` and `cnr-2000` datasets, which are the same used in the original reference paper [4].

For the `ASIC_320k` dataset, despite extreme row irregularity ($CV = 61.4$)[4], the Merge-Based approach failed to outper-

form the static scheduling approach past 8 threads (see Figure 2). This is attributed to the dataset's small density ($\approx 0.002\%$); the computational overhead of the merge-path binary searches outweighed the benefits of load balancing. Conversely, the `cnr-2000` dataset yielded a **1.65x speedup** over static scheduling within a single socket. However, performance for both algorithms degraded sharply beyond 24 threads. This inflection point corresponds exactly to the physical core count of a single socket. Scaling to a second socket forced remote memory accesses across the interconnect, saturating bandwidth and masking the algorithmic gains. In `termomech_dK.mtx`, the least irregular matrix, `schedule(static)` proves more efficient with respect to Merge-Based algorithm.

8. Conclusions

This work presented a comprehensive performance analysis of CSR SpMV parallelization strategies using OpenMP. Our theoretical and experimental evaluation demonstrates that the SpMV kernel has practical bottleneck limits and is strictly memory-bound. Consequently, we prioritize maximizing memory locality and load balance exploring custom-tailored solutions that exploit *omp pragma schedulings*.

We finally compare our initial results with a custom implementation of the state of the art Merge-Based CSR Spmv algorithm: we conclude that while Merge-Based effectively neutralizes load imbalance, its utility is bounded by problem size (overhead dominance) and hardware topology. For multi-socket scaling, the algorithm must be coupled with strict NUMA-aware memory allocation to prevent interconnect saturation.

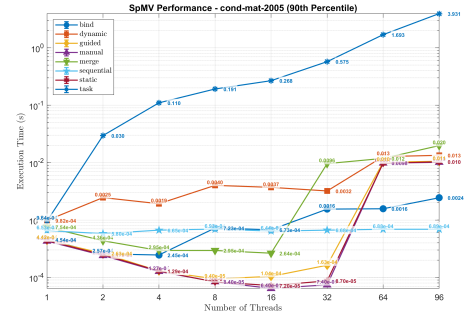


Figure 1. Execution time on `cond-mat-2005.mtx` with optimization flags (the most sparse matrix of the samples)

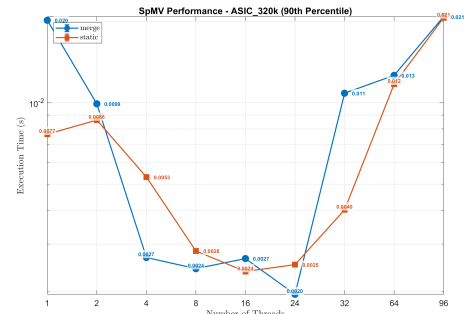


Figure 2. `spmv_static` vs `spmv_merge` on the most irregular sample matrix (`ASIC_320k.mtx`)

^aOn cache performance: for all the source code, `valgrind` has been used to detect changes in cache misses that could potentially indicate false sharing. Since the cache misses both in L1, L2 and L3 were practically equal in all cases; our hypothesis is that the matrices analyzed are small enough to fit in cache.

References

1. Gao J, Liu B, Ji W, and Huang H. A Systematic Literature Survey of Sparse Matrix-Vector Multiplication. arXiv e-prints 2024, arXiv:2404.06047:arXiv:2404.06047.
2. Appel AW. Formal Verification of COO to CSR Sparse Matrix Conversion (Invited Paper). arXiv e-prints 2025, arXiv:2510.13412:arXiv:2510.13412.
3. Buluç A, Fineman JT, Frigo M, Gilbert JR, and Leiserson CE. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In: *Proceedings of the Twenty-First Annual Symposium on Parallelism in Algorithms and Architectures*. SPAA '09. Calgary, AB, Canada: Association for Computing Machinery, 2009:233–44. doi: 10.1145/1583991.1584053. URL: <https://doi.org/10.1145/1583991.1584053>.
4. Merrill D and Garland M. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In: *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2016:678–89. doi: 10.1109/SC.2016.57.
5. Alexandersen J, Lazarov bibinitperiod, and Dammann B. Parallel Sparse Matrix - Vector Product: Pure MPI and hybrid MPI-OpenMP implementation. D T U Compute. Technical Report 2012–10. Technical University of Denmark, 2012.
6. Yzelman AJN and Roose D. High-Level Strategies for Parallel Shared-Memory Sparse Matrix-Vector Multiplication. *IEEE Transactions on Parallel and Distributed Systems* 2014;25:116–25.
7. Asudeh O, Saravani SM, Sabin G, Rastello F, and Sadayappan P. Is Sparse Matrix Reordering Effective for Sparse Matrix-Vector Multiplication? 2025. arXiv: 2506.10356 [cs.DC]. URL: <https://arxiv.org/abs/2506.10356>.
8. Bell N and Garland M. Implementing sparse matrix-vector multiplication on throughput-oriented processors. In: *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*. SC '09. Portland, Oregon: Association for Computing Machinery, 2009. doi: 10.1145/1654059.1654078. URL: <https://doi.org/10.1145/1654059.1654078>.